

Java Backend Architecture

Interview Series

Chapter 16.5

Docker Volumes & Storage

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 16.5 – Docker Volumes & Storage

Docker Storage Overview

Container filesystems are ephemeral by default — data is lost when a container is removed. Docker provides three storage mechanisms for persistent or shared data: named volumes, bind mounts, and tmpfs mounts. Choosing the right storage type is critical for data durability, security, and performance.

Storage Types Comparison

Docker Storage Options – Comparison

Named Volume	Bind Mount	tmpfs Mount
<code>/var/lib/docker/volumes/</code>	Host path mounted directly	In-memory only
Docker-managed	Developer controls path	Not persisted to disk
Persistent across restarts	Live code reloading	Fast I/O
Shareable between containers	Risk: container can write anywhere on host FS	Use for sensitive data (credentials, tokens)
Backup with <code>--volumes-from</code>		

`-v myvolume:/app/data` vs `-v /host/path:/app/data` vs `--tmpfs /app/tmp`

Named Volumes	<code>docker volume create mydata</code> . Stored in <code>/var/lib/docker/volumes/mydata/_data</code> . Managed by Docker — Docker controls path and lifecycle. Portable, can use volume drivers for remote storage (NFS, cloud). Best for: databases, persistent application data.
Bind Mounts	<code>-v /host/path:/container/path</code> or <code>--mount type=bind,src=/host,dst=/container</code> . Host path must exist. Container can read/write host filesystem. Use for: development (hot-reload), configuration files, log collection. Risk: container writes anywhere on host if path is misconfigured.
tmpfs Mounts	<code>--tmpfs /app/tmp</code> or <code>--mount type=tmpfs,destination=/app/tmp</code> . In-memory only — data is lost when container stops. Use for: sensitive data (credentials, tokens, session data), high-speed temporary storage, anything that must not touch disk.

Volume Commands

```
# Named volume usage
docker volume create mydata
docker run -v mydata:/app/data myapp
# Bind mount
docker run -v /host/config:/app/config:ro myapp
# read-only
docker run --mount type=bind,src=/config,dst=/app/config,readonly myapp
# tmpfs
docker run --tmpfs /app/tmp:rw,noexec,nosuid,size=100m myapp
# Inspect volumes
docker volume ls
docker volume inspect mydata
docker volume rm mydata
docker volume prune # remove unused volumes
```

Volume Backup & Restore

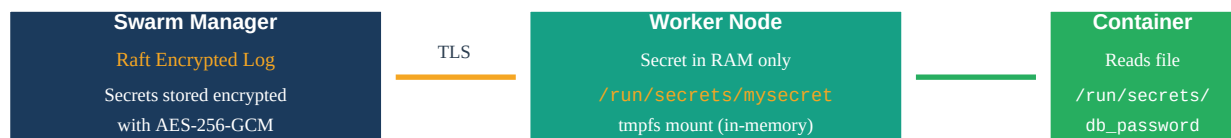
Volume Backup Workflow



```
# Backup: tar volume contents to host docker run --rm \ --volumes-from myapp_container \ -v $(pwd)/backup \ ubuntu tar czf /backup/mydata_backup.tar.gz /app/data # Restore: extract into new volume docker volume create mydata_restored docker run --rm \ -v mydata_restored:/data \ -v $(pwd)/backup \ ubuntu tar xzf /backup/mydata_backup.tar.gz -C /
```

Docker Secrets (Swarm)

Docker Swarm Secrets – Raft Encrypted Storage



Secret never written to disk on worker. Removed when service is updated/removed.

```
docker secret create db_password password.txt
```

```
# Create a secret echo "myS3cr3tP@ssw0rd" | docker secret create db_password - docker secret create tls_cert ./cert.pem # Use in a service docker service create \ --name myapp \ --secret db_password \ --secret source=tls_cert,target=/run/secrets/cert.pem,mode=0400 \ myapp:latest # In application code: read from file # Java: Files.readString(Path.of("/run/secrets/db_password")).trim()
```

Volume Drivers for Remote Storage

Driver	Use Case	Command
local (default)	Host disk storage	docker volume create mydata
nfs	Shared NFS storage	docker volume create --driver local --opt type=nfs --opt o=addr=nfsserver,rw --opt de
rclone	Cloud storage (S3, GCS, Azure)	docker volume create --driver rclone rclone-volume
REX-Ray	EBS, EFS, Azure Disk	StorageOS or Rex-Ray plugin
Portworx	Enterprise storage HA	Portworx plugin + StorageClass

Spring Boot External Config Volume

```
# Mount application.properties from host docker run -d \ -v /opt/myapp/config:/app/config:ro \ -e SPRING_CONFIG_ADDITIONAL_LOCATION=/app/config/ \ myapp:latest # Or override specific properties docker run -d \ -v /opt/myapp/application-prod.yml:/app/application.yml:ro \ myapp:latest
```

Interview Q&A; – Docker Volumes & Storage

Q1. What is the difference between named volumes and bind mounts?

Named volumes are managed by Docker — Docker creates and manages the storage location (`/var/lib/docker/volumes/`). They are portable, support volume drivers for remote storage, and can be safely shared between containers. Bind mounts directly mount a host filesystem path into the container. Docker doesn't manage them — the path must exist on the host. Bind mounts give the container access to host files, useful for development (hot-reload) and config injection. Named volumes are preferred for production data storage.

Q2. What is a tmpfs mount and what are its use cases?

tmpfs mounts store data in host memory — never on disk. Data is lost when the container stops. Use cases: (1) Sensitive credentials that must not be written to disk or appear in Docker logs/inspection. (2) Session tokens, encryption keys, one-time passwords. (3) High-speed temporary scratch space for data processing. (4) Test environments where ephemeral storage suffices. Configure size limit: `--tmpfs /tmp:rw,size=100m,noexec,nosuid` to prevent abuse. Kubernetes equivalent: `emptyDir` with `medium: Memory`.

Q3. How do Docker secrets work in Swarm mode?

Docker secrets are stored encrypted in the Raft log (AES-256-GCM encryption) on manager nodes. When a service is created with a secret, the Swarm manager sends the secret to worker nodes over a mutually authenticated TLS connection. On the worker node, the secret is stored in an in-memory tmpfs filesystem and mounted at `/run/secrets/secret_name`. When the service is removed or the secret is revoked, the secret is deleted from memory. Secrets are never written to disk on worker nodes and never appear in `docker inspect`.

Q4. What is --volumes-from and how do you use it for backups?

`--volumes-from` mounts all volumes from another container (by reference, not copy). The containers share the same volume mount points. Primary use: backup pattern — run a temporary backup container with `--volumes-from target_container` to access its volumes: `docker run --rm --volumes-from mydb -v $(pwd):/backup ubuntu tar czf /backup/db_backup.tar.gz /var/lib/postgresql/data`. This works without stopping the target container. Also used in the data container pattern (though named volumes are now preferred).

Q5. What happens to a named volume when a container is deleted?

Named volumes persist independently of container lifecycle. `docker rm container` does NOT delete associated named volumes. You must explicitly delete volumes: `docker volume rm myvolume` or `docker volume prune` (removes all unused volumes). This is intentional — data durability is the primary purpose of named volumes. Compare with anonymous volumes (created by `VOLUME` instruction in Dockerfile without a name) — `docker rm -v` removes anonymous volumes but NOT named volumes. Always use named volumes for data you care about.

Q6. How do you share a volume between multiple containers?

Named volumes can be mounted by multiple containers simultaneously: `docker run --name app1 -v mydata:/data myapp` and `docker run --name app2 -v mydata:/data myapp`. Both containers read/write the same underlying storage. For concurrent writes, the application must handle concurrency (file locking, database). Use cases: shared configuration files, shared upload directories, sidecar log collectors reading app logs. In Docker Compose: services define the same named volume and it's mounted to both.

Q7. What are the security risks of bind mounts?

Bind mounts give the container direct access to host filesystem paths. Risks: (1) Container escapes — if the bound path is `/`, the container can read/write the entire host filesystem. (2) Sensitive data exposure — binding `/etc` or `/var` gives container access to host credentials. (3) Container can overwrite host binaries if bound paths

include executables. Mitigations: use `:ro` (read-only) for config mounts, bind only specific directories, never bind root or system directories, use named volumes instead of bind mounts for production data.

Q8. How do you implement volume permissions for non-root containers?

Non-root containers (USER 1001) may not have write access to volume mount points owned by root. Solutions: (1) In Dockerfile: `RUN mkdir /app/data && chown 1001:1001 /app/data` — Docker creates the volume mount point with correct ownership. (2) Use an entrypoint script that `chown`'s the volume on startup (requires a brief root step). (3) Set VOLUME after USER in Dockerfile — Docker creates the directory with the current user. (4) In Kubernetes: use `fsGroup` in `securityContext` to set group ownership of mounted volumes.

Q9. What is the Docker volume inspection JSON and what fields are important?

`docker volume inspect myvolume` returns JSON with: Name (volume identifier), Driver (storage driver), Mountpoint (`/var/lib/docker/volumes/myvolume/_data` — actual host path), Labels (metadata), Scope (local vs global for cluster drivers), Options (driver-specific options like NFS server address). The Mountpoint is where you'd find the actual data files on the host for backup. UsageData shows size if docker system `df -v` was recently run. Status may include driver-specific health information.

Q10. What is the data container pattern and is it still recommended?

The data container pattern (pre-Docker 1.9) used a dedicated container with named volumes that never ran any process — just provided volumes via `--volumes-from` to other containers. Example: `docker create --name dbdata -v /var/lib/postgresql/data postgres:latest`. Other containers attach with `--volumes-from dbdata`. This pattern is now deprecated — named volumes (`docker volume create`) are the modern replacement. Named volumes are better managed, inspectable, have driver support, and don't require a dummy container.

Q11. How do you configure Docker volumes in docker-compose.yml?

In `docker-compose.yml`: define volumes at top level under 'volumes:' section, then reference them in services. Named volumes persist across docker compose down. To remove volumes: `docker compose down -v`. Example: `services: db: image: postgres: volumes: [pgdata:/var/lib/postgresql/data]. volumes: pgdata: (no driver = local default). External volumes (created outside compose): volumes: mydata: external: true. Volume names are prefixed with project name unless using external: true or a custom name.`

Q12. What is tmpfs and how does it differ from Docker tmpfs mounts?

Linux tmpfs is a virtual filesystem stored in RAM (and swap). Files in tmpfs are fast but lost on reboot/unmount. Docker `--tmpfs` mounts a tmpfs filesystem at a specific path inside the container. The data never touches the container's OverlayFS layer or the host disk — it exists only in the host's RAM allocated to that container. This is different from writing to the container layer (which goes to OverlayFS upperdir on disk). Key benefit: sensitive data (keys, tokens) cannot be recovered from disk forensics.

Q13. Explain the difference between VOLUME in Dockerfile and docker run -v.

VOLUME in Dockerfile declares a mount point and creates an anonymous volume at that path when the container starts (if no volume is mounted). Any data in the image at that path is copied to the anonymous volume at container creation. `docker run -v myvolume:/data` mounts a named volume or bind mount, overriding any VOLUME declaration. Important: VOLUME prevents writing to that layer — writes always go to the volume. This means: `RUN` commands after VOLUME cannot write to that path effectively. Define VOLUME last in Dockerfile.

Q14. How do you use Docker secrets outside of Swarm mode?

Docker secrets are a Swarm feature. For non-Swarm environments: (1) Bind mount a file: `-v /host/secrets/db_password:/run/secrets/db_password:ro` (same mount point pattern). (2) Use environment variables (less secure — visible in `docker inspect`, `ps env`). (3) Use a secrets manager client at startup: `fetch`

from AWS Secrets Manager, HashiCorp Vault, or Azure Key Vault, write to tmpfs. (4) Docker Compose supports secrets with file-backed secrets for local development: secrets: mysecret: file: ./secret.txt. Kubernetes uses K8s Secrets (base64 encoded, RBAC controlled, optionally with external secrets operator).

Q15. What is a volume mount in read-only mode and when would you use it?

-v /host/config:/app/config:ro mounts the volume in read-only mode. The container cannot write to or modify the mounted path. Use cases: (1) Configuration files (application.yml, nginx.conf) — prevent container from modifying its config. (2) TLS certificates — read but don't overwrite. (3) Shared reference data that multiple containers should read but not modify. (4) Source code mount in read-only dev containers. In Compose: volumes: - ./config:/app/config:ro. Combining with --read-only filesystem provides strong write protection.

Q16. How do volume drivers enable cloud storage integration?

Volume drivers are plugins that enable Docker volumes to use remote or distributed storage backends. Popular drivers: REX-Ray (EBS, EFS, Azure Disk, GCE PD), rclone (S3, GCS, Azure Blob, 30+ backends), Portworx (enterprise, HA, encryption, snapshots). Install: docker plugin install portworx/...; Create: docker volume create --driver pxd --opt size=100 --opt repl=2 mydata. The container uses -v mydata:/data normally — the driver handles remote mounting. This enables stateful containers that can be rescheduled across hosts while retaining data.

Q17. What is the difference between volumes and bind mounts in terms of performance?

Named volumes (overlay2, local driver): data stored on host filesystem, but access goes through Docker's volume management layer. Performance is similar to native filesystem. Bind mounts: direct host filesystem access, no intermediary — fastest for I/O (database workloads). tmpfs: fastest of all (RAM), but volatile. On Docker Desktop (Mac/Windows): bind mounts cross a VM boundary (VirtioFS on Mac) which adds latency. Named volumes on Docker Desktop are managed inside the VM and are faster than bind mounts. On Linux, performance difference between named volumes and bind mounts is negligible.

Q18. What happens to volume data during a container image upgrade?

Volumes are independent of images. When you docker pull newimage:2.0 and run a new container with the same volume, the data persists unchanged. The volume from the old container is mounted into the new container. Consideration: database migrations — if the new app version expects a different schema, you must run migrations before switching. Pattern: (1) Stop old container. (2) Backup volume. (3) Run migration container on same volume. (4) Start new app container. Kubernetes handles this with init containers that run migrations before the app starts.